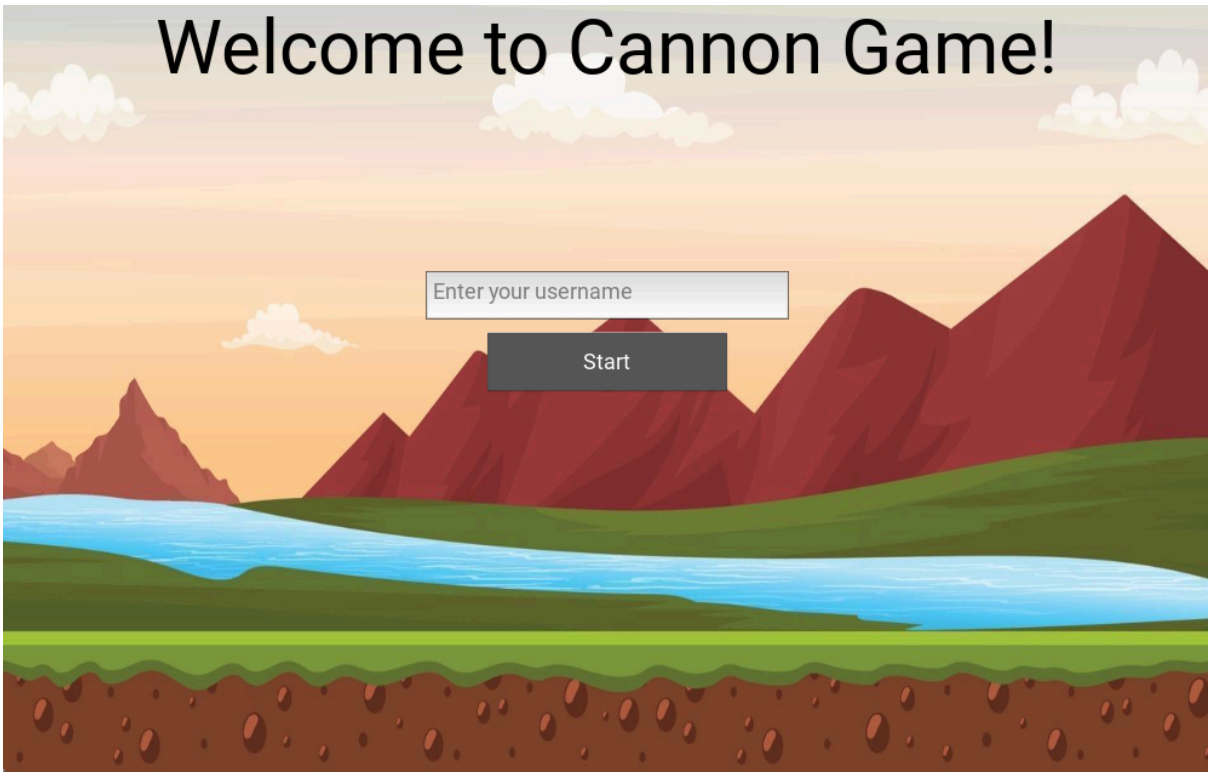
THE CANNON GAME



Final Exam Project

Computer Programming, Algorithms and Data Structures, Mod. 1
September 24, 2025

Daria Miele (mat. 535702)

Academic Year: 2023/2024

1. Introduction

1.1 Project Overview: description of the objectives and constraints

Cannon Game is a 2D physics-based artillery game developed using **Python** and the **Kivy** framework. The core mechanic consists of shooting different types of projectiles using a cannon located on the left side of the screen, to hit a target placed randomly on the opposite side. The game is played through ten rounds of increasing difficulty, where obstacles such as rocks, perpetio, bulletproof mirrors and elastonio surfaces challenge the path of the projectiles and require strategic thinking to succeed.
Each round begins by generating a new target and a set of obstacles, and it ends as soon as the target is hit. The player has **up to five attempts per round**, after which the game is lost if the target has not been hit. Otherwise, the user wins the game if it successfully hits the final target in round 10: the overall objective of the player is to **complete all the rounds using as few shots as possible**, since the total number of shots determines the final score.

In developing this project, the main goal was to combine programming basics with applied concepts of physics and game design. The focus was given to simulating projectile motion using simplified physical laws: parabolic trajectories for massive projectiles and linear trajectories for lasers, as well as implementing unique interactions with the obstacles.
The project was also an opportunity to deepen the understanding of **object-oriented programming**, modular software design, and Kivy's event-driven framework.
At the beginning, the construction of an entire interactive application in Kivy was one of the main problems to face, as I was accustomed to more traditional Python projects. To overcome this hurdle, I decided to gradually build the game in steps: I started implementing the basic screens with just the cannon, then added a single projectile type (bullet). After that I expanded to multiple projectile types and obstacles. In this way, I was able to steadily improve both the technical implementation and the overall gameplay experience.

In order to create an engaging environment and a coherent interface, widgets, labels and buttons were designed and integrated with background images and graphics. The graphics might seem relatively minimalistic, however the focus was to ensure functionality and clarity of the interface. The inclusion of additional features such as **Help**, **Hall of Fame**, and **Save/Load** helped to evolve the project from just a basic prototype into a complete and replayable game.

The main objectives of the Cannon Game can be summarized as follows:

- **Implementing a physics engine** to simulate projectile trajectories under gravity, linear laser motion, and their interaction with obstacles.
- **Developing a progressive difficulty system** by introducing increasingly complex obstacles across the ten rounds.
- **Creating a clear and intuitive interface**, enabling players to adjust the angle and velocity with sliders, switch between projectile types, and easily navigate between different screens.

2. Game Specifications

2.1 Core elements of the gameplay

The game is structured around several **core components** that define its mechanics.

- A **Cannon** controlled by the player, which can be adjusted both in firing angle and muzzle velocity;
- A **Target** that changes position at each round;

- **Obstacles** that **block, reflect,** or **bounce** projectiles, altering their trajectories.
- **Projectile physics** incorporating gravity, reflection, and penetration mechanics.

2.2 Projectiles

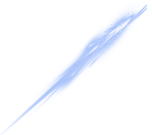
The game includes different kinds of projectiles, each with its own physical behaviour.
They move according to simplified physics: massive projectiles follow a simplified parabolic trajectory under constant gravity, while the laser is massless and is not affected by gravity at all: it travels in a straight line with constant speed, for a limited amount of time.

Three types of projectiles were implemented:

- **BULLET**: massive projectile subject to gravity. It follows a parabolic trajectory and has a limited impact radius (`BULLET_RADIUS = SCREEN_WIDTH/70`);

- **BOMBSHELL**: massive projectile subject to gravity. It follows a parabolic trajectory too, but it has a larger explosion radius (`BOMB_RADIUS = SCREEN_WIDTH/50`);

- **LASER**: massless, it is not affected by gravity and follows a linear trajectory with a limited duration.



2.3 Obstacles

The projectile can interact with environmental obstacles that alter, reflect or block its path. Four kinds of obstacles are implemented: each interacts differently with the projectiles, thus increasing the difficulty across rounds.

- **ROCK**: ground element that is destroyed if hit by any projectile;

- **PERPETIO**: indestructible ground element, it blocks the projectiles.

- **BULLETPROOF MIRROR**: reflects laser projectiles at an angle based on the impact direction. It may move after a certain number of rounds;

- **ELASTONIO**: reflects massive projectiles with an elastic effect, it may deform slightly and animate after a certain number of rounds.

2.4 Game Functionalities

The game provides the following key features to enhance the user's experience:

- **Help screen** to provide basic instructions to guide the player;
- **Hall of Fame** to register best scores: it is a persistent leaderboard based on fewest total shots;
- **Save/Load** of game progress, to store round, attempts and selected projectiles.

2.5 Visualization

The final implementation uses a **single fixed window visualization**, meaning that the entire playing field is always visible to the player without the need for scrolling. The game was designed for a fixed resolution of 1000x700 pixels, chosen to simplify the placement of graphic elements and maintain a consistent user interface. Full-screen support was excluded to avoid issues with dynamic scaling and to ensure stable game behavior.

This choice ensures that the cannon, the target, and all obstacles are simultaneously displayed, allowing the player to plan each shot while fully aware of the environment.
The cannon is placed on the left side of the screen, while the target appears on the right side in different positions throughout the rounds. Obstacles are placed between the cannon and the target, forcing the player to carefully choose both angle and velocity when firing.

The background remains the same across the game, but the progression of rounds is visually marked by the increasing number and variety of obstacles.
The user interface always provides clear and essential information. A **status label** shows the current round, the number of attempts already used, and the maximum allowed per round. At the end of the game, whether in case of victory or game over, additional labels and buttons appear to inform the player of the result and to allow restarting or saving the score in the Hall of Fame.

The design also includes **control elements** to make gameplay interactive and flexible: the player can adjust the cannon angle and the muzzle velocity using sliders, while projectiles are fired with the spacebar. A dedicated screen allows the players to choose the projectile type both before and during the game, as long as they remain within the five attempts per round. These features make the interface intuitive and responsive, while preserving the challenge and strategy typical of artillery games.

3. Implementation

3.1 General Overview

The game takes place in a two-dimensional space, with lengths measured in pixels, time in seconds, and consequently velocity in pixels per second. The game is updated at each frame using Kivy's Clock, with a frame rate of about 60 updates per second, which ensures both smooth animation and an accurate approximation of continuous motion.

3.2 Modular design and logical structure

The logical structure of the solution is based on **modularization**, meaning that each entity of the game is encapsulated in its own class. This choice made the code easier to read and debug. It also made the game naturally extensible: allows to add new obstacle types or projectile behaviors in the future.

The `CannonApp` class is the starting point of the application: it initializes the `ScreenManager`**,** which handles navigation across different screens (Welcome, Projectile Selection, Gameplay, Help and Hall Of Fame). The game flow follows this modular design: at startup the player is greeted by the Welcome screen, then proceeds to Projectile Selection, enters the main gameplay loop, and access the Help or Hall of Fame screens when needed.
From an implementation perspective, the Kivy framework provided all the essential tools to build this flow. The use of **widgets** was fundamental in the representation of the game entities (cannon, projectiles, obstacles, and target) as well as the interface elements (labels, sliders, and buttons). Positioning of widgets and screen switching were dealt with **layout management**, whereas **event binding** allowed user interaction: sliders for adjusting the cannon's angle and velocity, button clicks for navigation or projectile changes, and the spacebar for firing the projectile.

4. Code Explanation

4.1 Constants

All simulation parameters are defined at the beginning of the code as constants, ensuring flexibility and easy tuning of the game physics:

- **Screen dimensions**: `SCREEN_WIDTH`, `SCREEN_HEIGHT`
- **Frame rate**: managed by `Clock.schedule_interval(update, 1/60)`
- **Projectile parameters**:
  Bullet: `BULLET_RADIUS`, `BULLET_MAX_VEL`
  Bomb: `BOMB_RADIUS`, `BOMB_MAX_VEL`
  Laser: `LASER_IMPULSE`, `LASER_VEL`
- **Gravity**: applied only to bullet and bombshell projectiles.

4.2 User interface and screens

The user interface is managed using Kivy's `ScreenManager` with multiple screens: each one encapsulates its own widgets (labels, buttons, sliders, backgrounds).

- **WelcomeScreen**: allows entering a username and navigating to game, Help, or Hall of Fame.
- **ProjectileSelectScreen**: dedicated screen for choosing between bullet, bomb, or laser.

- **CannonGame** (game screen): main gameplay screen with cannon, target, projectiles, obstacles, and status labels.
- **HelpScreen**: displays instructions and controls.
- **HallOfFameScreen**: shows saved scores from previous matches.

4.3 CannonGame(Screen) Class

This is the **core loop of the game**, responsible for managing gameplay. It manages:

- **Projectile firing**: instantiate and launch projectile via `fire_projectile()`.

- **User input**: sliders for angle/velocity, spacebar for firing, buttons for navigation.

- **Round management**: create new targets and obstacles (`new_round()`, `next_round()`).

- **Collision detection**: `check_collision()` verifies contact between projectiles, obstacles, and target. Specialized behavior depending on projectile type (bullet radius, bomb explosion radius, laser impulse).

- **Game flow**: handle game over, win condition, saving progress and updating labels.

4.4 Projectile(Image) Class

Represents the fired projectiles, encapsulating physics and behavior of all projectile types. This unified class avoids code duplication while still distinguishing the behavior of each type.

- **Trajectory**
  Bullet and bomb: parabolic motion under gravity.
  Laser: linear motion at constant speed, unaffected by gravity, limited duration.

- **Physics properties**: each projectile stores velocity components (`vx`, `vy`) and angle.

- **Collision response**:
  Bullet checks collisions via a limited impact radius.
  Bombshell triggers explosion animation and has a larger explosion radius .
  Laser rotation is handled via canvas transformation (`Rotate`).

4.5 Obstacle Classes

Each obstacle is encapsulated in its own class, inheriting from `Image`, with its own **on_hit** or **update** method to define how it interacts with projectiles.

- **RockObstacle**: destructible, removed from the screen when hit.
- **PerpetioObstacle**: indestructible, removes projectiles that collide with it.
- **MirrorObstacle**: reflects lasers according to reflection laws, can move horizontally if enabled.
- **ElastonioObstacle**: deforms elastically when hit by bullets or bombs and reflects them with restitution factor, but absorbs lasers. It can move vertically in later rounds.

4.6 Game Flow

The flow of the game ensures a smooth experience, to ensure that every element of the game is connected and updated efficiently.

1. **Startup** → Welcome screen: the user chooses whether to start, view help, or see the Hall of Fame.

2. **Projectile selection** → Player chooses the type of projectile to play with.

3. **Gameplay loop** → The user fires the cannon through the space bar. Cannon fires, projectiles move, collisions with obstacles and target are checked.

4. **Round result** → If the target is hit, `next_round()` starts a new round; if maximum attempts are used without success, `game_over()` is triggered.

5. **Endgame** → After round 10, the score is saved in the Hall of Fame.

Throughout gameplay, the **update loop** ensures smooth animations and consistent physics, while the UI provides constant feedback on round, attempts, velocity, and angles.

5. Tools and resources

5.1 Libraries

The following libraries were used in the project:

- **Python 3.12** – the programming language used for logic, physics, and data handling.
- **Kivy 2.3.0** – framework for building the interface, managing screens, rendering widgets, and handling real-time updates.
- **Math** (Python standard library) – for trigonometric and geometric calculations (trajectory, collision detection, reflection).
- **Random** (Python standard library) – for random generation of target and obstacle positions, ensuring variability.
- **JSON** (Python standard library) – for progress saving and score storage in a simple, portable format.

5.2 References and learning sources

[1] Python (v3.12)

[2] Kivy (v2.3.0)

[3] VS Code (v1.104.0)

[4] Official Kivy documentation: https://kivy.org/doc/stable

[5] Kivy GitHub repository: https://github.com/kivy/kivy

[6] Python standard library documentation: https://docs.python.org/3/library/

[7] FreeCodeCamp, Kivy Course: https://www.youtube.com/watch?v=I8Imtec4ReQ

[8] Amanda Hogan, Kivy Basics: https://www.youtube.com/watch?v=3GBNMBhm6UU.

[9] Chat GPT-4o

[10] 2023_24_cannon.pdf

6. Testing and results

Testing was necessary to ensure that the game behaved as expected under the specified rules of physics, round progression, and user interaction. I choose to follow an **incremental testing strategy**, where functionalities were introduced one at a time and verified before moving on to the next feature.

In fact, initially the main focus was only on the **essential structure** of the game: the cannon, one basic projectile (bullet), and a static target. This allowed to check that the projectile motion followed the correct equations and that the player could control angle and velocity and fire projectiles as intended. Once this core mechanics was confirmed, **further projectile types were gradually integrated**: in particular, the bomb was tested for its larger impact radius, while the laser was validated for its linear, gravity-free trajectory and limited impulse duration. When all projectiles were working properly, obstacles were introduced: **each obstacle type was tested independently** to ensure that its unique interaction rules were correctly implemented.

Then, **round progression** was also tested: each round had to generate a fresh target and a growing number of obstacles, placed randomly but without overlapping each other. Later rounds introduced moving obstacles, which were tested for smooth movement and consistent collision detection.
Game Over and Win conditions were simulated to verify the correct display of interface elements, reset functionality, and saving the score in the Hall of Fame; the game was able to store attempts, level, and projectile type in a JSON file and reload them without issues.

Overall, testing outcomes revealed that the game works consistently within the defined constraints. From the visual aspect projectiles move in a continuous manner, while on the technical side collisions are correctly detected, ensuring that the next round will be a step towards a gradual rise in difficulty. The inclusion of various obstacles and projectiles enriched the gameplay experience, while the user interface provided clear and intuitive feedback. Some adjustments were made as a result of tests, for instance changing projectile obstacles speed to match the level of the challenge and gameability.

7. Possible improvements

Although the game is functional and meets the project requirements, there are several ways it could be improved in the future.

- **Visual design** : right now, the game uses simple images for the cannon, projectiles, obstacles, and background. Adding custom graphics, animations, or particle effects would make the game more visually engaging. Sound effects and background music could also be added to make the gameplay more immersive.
- **Gameplay variety** : at the moment, the player can choose between three projectile types and faces a limited set of obstacles. It would be interesting to introduce new projectile mechanics or obstacle types.
- **Enhanced obstacle behavior** : obstacles like the Mirror or Elastonio already have interesting interactions, but more complex behaviors could be added. For instance, rotating mirrors that continuously change their angle, or elastic platforms that launch projectiles in unpredictable ways.
- **Advanced scoring system** : instead of only tracking total shots, the score could also consider accuracy, time taken, or bonus points for special shots.

8. Conclusion

Developing the Cannon Game was a complex process that involved both technical skill and learning. This project required combining principles of object-oriented programming, physics simulation, and the design of user interface in the Kivy framework. By using modular classes for the cannon, projectiles, obstacles, and screens to structure the game, a clean and extensible solution was created.

The achieved result is a functional game that balances strategy and fun: players are required to adjust the angle, velocity, and projectile type in a very precise manner in order to hit the target, at the same time they have to be able to adapt to the obstacles that become increasingly complex with each new round. The multiple screens, the ability to save progress, and the Hall of Fame are features that give an overall user experience and well-structured game qualities.
Beyond the coding itself, the project highlighted the importance of incremental development, testing at each stage and adapting design choices when initial solutions were not successful. The game, while it already meets its core objectives, also opens up the possibility of many extensions due to its modular design. This makes the Cannon Game not just a finished product, but also a solid foundation for future development.